

Laborator 6 – Utilizarea bibliotecilor de funcții

6.1 Scopul lucrării

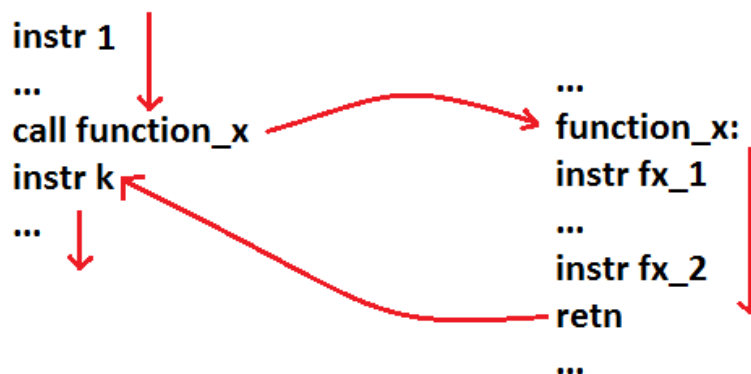
În această lucrare de laborator, se va discuta utilizarea funcțiilor de bibliotecă. Principalele convenții de apel, pentru aceste funcții sunt cdecl, stdcall și fastcall. Biblioteca msvcrt pune la dispoziție funcțiile standard, întâlnite în limbajul C, dintre care se vor prezenta cele pentru afișare pe ecran, citire de la tastatură, respectiv citire/scriere din/în fișiere.

6.2 Rolul sistemului de operare și al bibliotecilor de funcții

Deși limbajul de asamblare folosește în mod direct componentele hardware ale sistemului, există porțiuni de cod utilizate frecvent, care ar fi impractic să fie scrise de către programator de fiecare dată. În plus, comunicarea cu dispozitivele de intrare/ieșire presupune de cele mai multe ori protocoale complexe, a căror utilizare necorespunzătoare poate duce inclusiv la avarii fizice ale acestora. Unul dintre rolurile sistemului de operare este acela de a abstractiza mașina hardware, pentru programator. Din acest motiv, un program ce rulează în spațiu utilizator nu va accesa direct dispozitivele de intrare/ieșire (limbajul de asamblare permite acest lucru, prin instrucțiunile `in` și `out`), ci va apela la sistemul de operare.

Deasemenea, pentru anumite operații frecvente, cum ar fi afișarea datelor într-un anumit format, găsirea unui subșir într-un șir sau diverse funcții matematice, există biblioteci de funcții, ce pot fi apelate.

Utilizarea unei funcții presupune saltul la porțiunea de cod corespunzătoare funcției, execuția acestui cod, urmată de revenirea la instrucțiunea de după cea care a apelat funcția, ca în figura de mai jos:



6.3 Utilizarea funcțiilor externe. Convenții de apel

Pentru apelul unei funcții se folosește instrucțiunea **CALL**. Această instrucțiune pune adresa instrucțiunii următoare pe stivă, apoi, sare la începutul funcției apelate. Punerea pe stivă a adresei instrucțiunii următoare (adresa de revenire) se face pentru ca la finalul execuției să se poată reveni la codul apelant. Se poate considera că apelul de funcție (CALL) și saltul necondiționat (JMP) sunt similare, cu excepția faptului că apelul de funcție pune pe stivă adresa de revenire.

De cele mai multe ori, funcțiile pot primi parametri, și pot întoarce rezultate. Există mai multe convenții pentru a face aceste lucruri, dintre care vom discuta 3 în acest laborator: cdecl, stdcall și fastcall. Pentru a exemplifica, vom considera următoarea funcție, dintr-un limbaj de nivel înalt:

```
int myfunc(int x, int y, int z, int t)
```

Vom dori să apelăm această funcție, cu parametrii a, b, c, și d, obținând rezultatul în variabila res:

```
res = myfunc(a, b, c, d)
```

Trebuie remarcat că o convenție de apel nu ține de sintaxa limbajului de asamblare, ci este un ”contract” între autorul funcției și utilizatorii acesteia, ce specifică modul de transmitere a parametrilor, respectiv de întoarcere a rezultatului. Cine scrie o funcție poate folosi orice convenție de apel dorește, putând inclusiv să inventeze una proprie. Este important în schimb, ca dacă funcția este apelată de altcineva, acesta să cunoască convenția de apel folosită. În limbaj de asamblare, nu este nevoie să se specifice asamblorului, convenția folosită.

6.3.1 Convenția cdecl

La această convenție, argumentele funcției se vor pune pe stivă, în ordine inversă (de la dreapta la stânga), iar rezultatul va fi returnat în registrul EAX. Registrii EAX, ECX și EDX pot fi folosiți în interiorul funcției (acest lucru înseamnă că după apel, valorile acestora pot diferi față de starea dinaintea apelului). Funcțiile ce folosesc convenția cdecl nu vor curăța argumentele de pe stivă, această sarcină revenind apelantului.

O excepție de la regula de returnare a rezultatului apare la funcțiile care returnează un rezultat în virgulă flotantă. Acest rezultat nu se va mai pune în registru EAX, ci în registru flotant ST(0).

De exemplu, codul în asamblare, pentru a face apelul de mai sus, este:

1	push d
2	push c
3	push b
4	push a

5	call myfunc
6	add esp, 12
7	mov res, eax

La liniile 1-4 se pun pe stivă argumentele funcției (presupunem că variabilele a, b, c, d, res au fost declarate de tip DWORD). Urmează apelul funcției la linia 5, apoi curățarea stivei, la linia 6. Pentru a curăța stiva trebuie să adunăm la registrul ESP, ce reține adresa vârfului, lungimea totală a datelor puse pe stivă (deoarece atunci când punem ceva pe stivă, adresa vârfului descrește). Am pus pe stivă 3 variabile de tip DWORD, ce ocupă fiecare 4 octeți, deci trebuie să adunăm 12. Nu în ultimul rând, rezultatul obținut în registrul EAX, va fi mutat în variabila rez, la linia 7.

Ca exemplu de funcții ce respectă convenția cdecl, avem funcțiile standard din limbajul C (de exemplu printf).

6.3.2 Convenția stdcall

Această convenție de apel este similară cu cdecl, diferența fiind că sarcina de a curăța argumentele de pe stivă revine funcției apelate, nu apelantului. Această convenție de apel este potrivită doar pentru funcțiile cu număr fix de parametri.

Pentru exemplul de mai sus, apelul se face în modul următor:

1	push d
2	push c
3	push b
4	push a
5	call myfunc
6	mov res, eax

Observăm că singura diferență apare la linia 6, renunțându-se la curățarea stivei.

O convenție similară cu stdcall este convenția pascal, diferența fiind că argumentele se vor pune pe stivă în ordine, începând cu argumentul cel mai din stânga.

Convenția stdcall este specifică API-urilor Win32 (funcții externe, oferite de sistemele de operare Windows pe 32 bit).

6.3.3 Convenția fastcall

Convenția fastcall (în varianta Microsoft) presupune transmiterea primilor 2 parametri ai funcției, de la stânga la dreapta, care încap ca reprezentare într-un DWORD, în regiștrii ECX și EDX, restul parametrilor punându-se pe stivă, de la dreapta la stânga. Rezultatul se va returna în registrul EAX (cu excepția rezultatelor în virgulă flotantă), iar stiva va fi curățată de funcția apelată, la fel ca la convenția stdcall.

1	mov ecx, a
2	mov edx, b
3	push d
4	push c
5	call myfunc
6	mov res, eax

Observăm transmiterea diferită a parametrilor, în liniile 1-4.

Avantajul acestei convenții de apel este viteza. Citirea unor date din regiștri este mult mai rapidă decât citirea din memoria principală.

6.4 Funcții standard din msvcrt

6.4.1 Afișarea pe ecran și citirea de la tastatură

Pentru afișarea unui text pe ecran, ce respectă un anumit format, se folosește funcția printf:

```
int printf ( const char * format, ... );
```

Primul argument al funcției este un string ce conține formatul afișării, urmat de un număr de argumente egal cu cel specificat în cadrul formatului. String-ul transmis în parametrul format poate conține anumite marcaje de formatare, ce încep cu caracterul '%', care vor fi înlocuite de valorile specificate în următoarele argumente, formatare corespunzător.

Formatul complet al unui astfel de marcator este:

```
%[flags][width][.precision][length]specifier
```

Specificatorul poate fi unul dintre caracterele:

specifier	Ce se afișează	Exemplu
c	Caracter	a
d sau i	Întreg zecimal cu semn	392
e	Notăție științifică (mantisă/exponent) folosind caracterul 'e'	3.9265e+2
E	Notăție științifică (mantisă/exponent) folosind caracterul 'E'	3.9265E+2
f	Număr întreg cu zecimale	392.65
g	Cea mai scurtă afișare dintre %e și %f	392.65
G	Cea mai scurtă afișare dintre %E și %f	392.65
o	Număr în octal, fără semn	610
s	String (șir de caractere, terminat cu 0)	sample
u	Întreg zecimal fără semn	7235
x	Întreg hexazecimal fără semn	7fa
X	Întreg hexazecimal fără semn (cu literele A-F mari)	7FA
p	Pointer	B800:0000
%	Două caractere '%' consecutive vor afișa unul la ieșire	%

Funcția printf respectă convenția cdecl, deci argumentele sale se vor transmite prin stivă, de la dreapta la stânga, iar curățarea acesteia va cade în sarcina apelantului.

Exemplu de program care afișează un întreg și un string:

1	.data
2	nume DB "Ion", 0
3	varsta DD 20
4	format DB "Ma numesc %s si am %d de ani.", 0
5	.code
6	start:
7	push varsta
8	push offset nume
9	push offset format
10	call printf
11	add esp, 12
12	push 0
13	call exit
14	end start

Pentru a citi date de la tastatură se folosește funcția scanf:

```
int scanf ( const char * format, ... );
```

Sintaxa acestei funcții este similară cu cea a funcției printf. Diferența majoră constă în faptul că argumentele sale nu trebuie să fie valori, ci adrese în memorie, unde se vor stoca valorile citite.

Codul de mai jos va afișa mesajul "n=", apoi va citi de la tastatură valoarea numărului n.

1	.data
2	msg DB "n=", 0
3	n DD 0
4	format DB "%d", 0
5	.code
6	start:
7	push offset msg
8	call printf
9	add esp, 4
10	push offset n
11	push offset format
12	call scanf
13	add esp, 8
14	push 0
15	call exit
16	end start

6.4.2 Citirea și scrierea din/în fișiere text

Atunci când utilizăm biblioteca msvcrt, conceptul de fișier text este similar cu cel din limbajul C. Un fișier se va deschide, se vor efectua operații de citire sau scriere, asupra lui, apoi se va închide.

Deschiderea și închiderea unui fișier se fac folosind funcțiile fopen și fclose.

```
FILE * fopen ( const char * filename, const char * mode );
```

```
int fclose ( FILE * stream );
```

La fel ca celelalte funcții specifice limbajului C, fopen și fclose respectă convenția cdecl.

fopen va primi ca parametri un string cu numele fișierului ce trebuie deschis și un string cu modul de deschidere. Rezultatul returnat va fi un pointer spre fișierul deschis.

Cele mai comune valori pentru modul de deschidere sunt:

mode	Utilizare
r	Deschidere în mod citire, pentru fișiere text
rb	Deschidere în mod citire, pentru fișiere binare
w	Deschidere în mod scriere, pentru fișiere text
wb	Deschidere în mod scriere, pentru fișiere binare
a	Deschidere în mod citire, adăugare

Codul de mai jos va deschide fișierul "fisier.txt" în mod citire, apoi îl va închide.

1	.data
2	mode_read DB "r", 0
3	file_name DB "fisier.txt", 0
4	.code
5	start:
6	push offset mode_read
7	push offset file_name
8	call fopen
9	add esp, 8
10	push eax ;in eax a fost returnat pointer-ul spre fișier
11	call fclose
12	add esp, 4
13	push 0
14	call exit
15	end start

Pentru a citi/scrie un fișier în mod text, se folosesc funcțiile fprintf și fscanf:

```
int fprintf ( FILE * stream, const char * format, ... );
```

```
int fscanf ( FILE * stream, const char * format, ... );
```

Utilizarea acestora este similară cu printf și scanf, excepția fiind primul parametru, ce trebuie să fie un pointer spre un fișier deja deschis.

În cazul unui fișier binar (caz general, ce poate include și fișiere text), funcțiile folosite sunt fread și fwrite:

```
size_t fread ( void * ptr, size_t size, size_t count, FILE * stream );
```

```
size_t fwrite ( const void * ptr, size_t size, size_t count, FILE * stream );
```

Spre deosebire de fprintf și fscanf, acestea nu citesc din fișier date cu un anumit format, ci vor citi conținut ”pur” binar.

Primul parametru reprezintă adresa de început în memorie, a unui buffer ce va fi citit/scriș. Al doilea parametru este dimensiunea unității de scriere. Dacă vrem să scriem câte un BYTE, valoarea va fi 1, pentru WORD 2, iar pentru DWORD 4. Al treilea parametru va fi numărul de elemente, de dimensiune size, ce vor fi scrise. Ultimul parametru este un pointer spre un fișier ce a fost deschis în prealabil în mod corespunzător.

6.5 Mersul lucrării

1. Discuții legate de problemele întâmpinate în înțelegerea documentației scrise.
2. Se va studia exemplul s6ex1.asm. Acesta afișează pe ecran un mesaj, folosind funcția printf, și diverse moduri de formatare. Programul va fi asamblat cu MASM, execuția sa se va trasa în Olly Debugger, urmărind în mod special stiva. Deasemenea programul se va rula în consolă, și se va observa rezultatul afișării.
3. Să se scrie în limbaj de asamblare un program care cere utilizatorului 2 numere, de la tastatură, apoi afișează suma acestora.
4. Se va studia exemplul s6ex2.asm, care afișează conținutul unui fișier binar în hexazecimal.
5. Să se citească de la tastatură un șir de caractere, și să se scrie într-un fișier text, șirul inversat.